

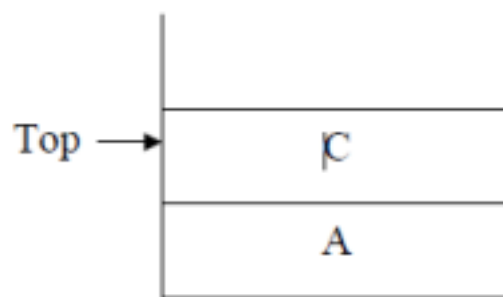
CS3301 DATA STRUCTURES

UNIT II STACKS AND QUEUES

Stack ADT – Operations – Applications – Balancing Symbols – Evaluating arithmetic expressions- Infix to Postfix conversion – Function Calls – Queue ADT – Operations – Circular Queue – DeQueue – Applications of Queues. **2.1. THE STACK ADT**

Stack Model :

- A stack is a linear data structure which follows Last In First Out (LIFO) principle.
- Insertion and deletion occur at only one end called the Top.



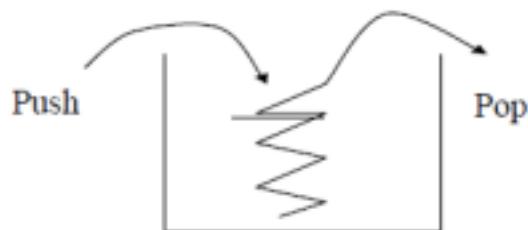
Stack Model

Examples : Pile of coins., a stack of trays in cafeteria

2.2. OPERATIONS ON STACK

The fundamental operations performed on a stack are

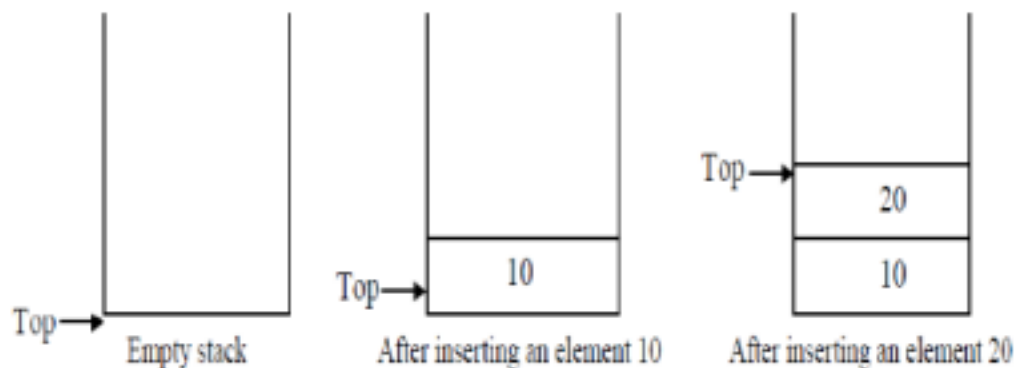
1. Push
2. Pop



Operations on stack

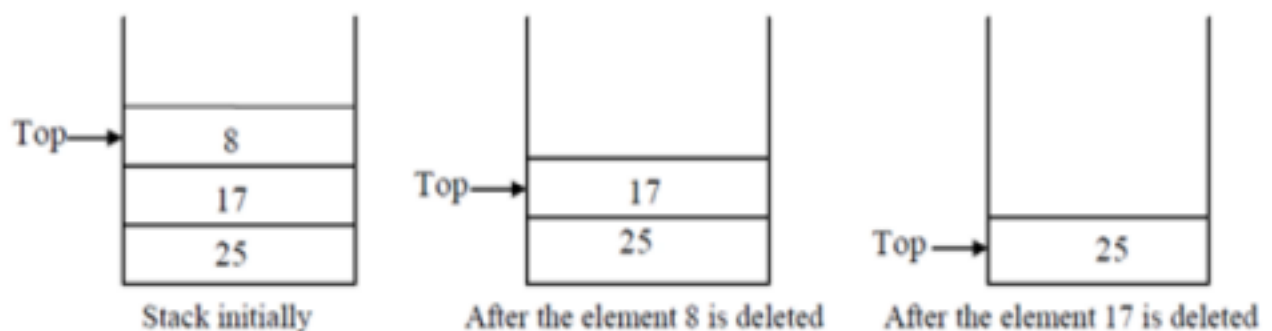
1. PUSH :

- The process of inserting a new element to the top of the stack. •
- For every push operation the Top is incremented by 1.



2. POP :

- The process of deleting an element from the top of stack is called pop operation.
- After every pop operation the Top pointer is decremented by 1.



Conditions

Overflow

Attempt to insert an element when the stack is full is said to be overflow.

Underflow

Attempt to delete an element, when the stack is empty is said to be

underflow **IMPLEMENTATION OF STACK ADT**

Stack can be implemented using

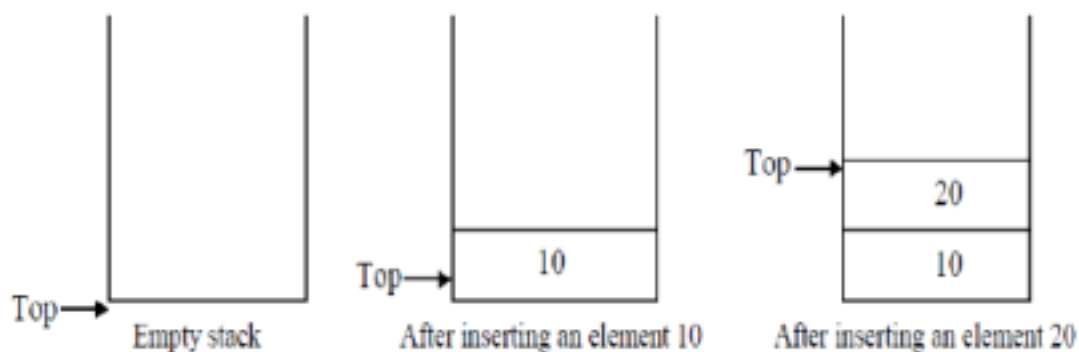
1. Array
2. Linked List .

Array Implementation

- Each stack is associated with a Top pointer.
- $\text{Top} = -1$ for an empty stack.
- To insert an element(PUSH),
 - Top is incremented by 1.

- Store the element in position Top. (stack[Top]=element).
- Push on a full stack is overflow.
- To delete an element(POP),
 - The stack [Top] value is returned.
 - Top pointer is decremented by 1.
 - Pop on an empty stack is underflow.

a) PUSH



Routine to push an element into stack

```
void push (int X, Stack S)
```

```
{  
    if (IsFull (S))  
        printf ("Full Stack");  
    else  
    {  
        Top = Top + 1; S[Top] = X;  
    }  
}
```

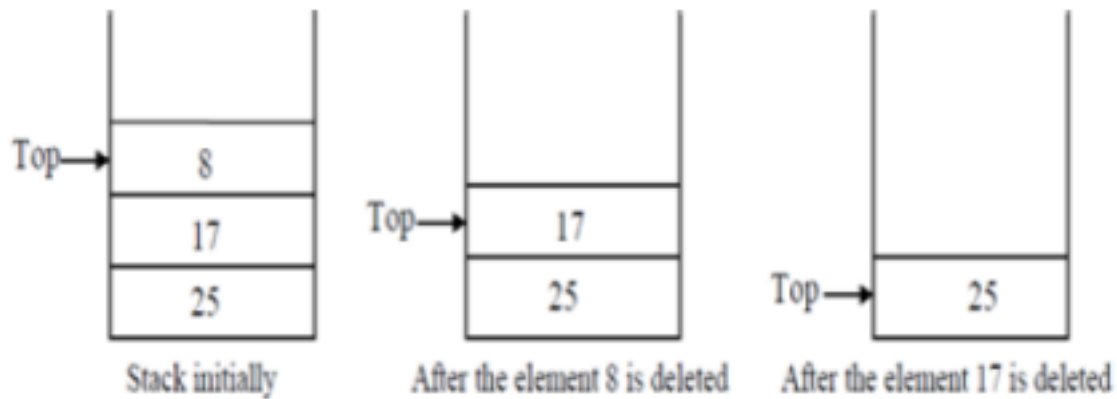
Routine to check full stack

```
int IsFull (Stack S)
```

```
{  
    if (Top == Stacksize-1)  
        return (1);  
    else  
        return(0);  
}
```

}

b)POP



Routine to pop an element from stack

void pop (Stack S)

```
{  
    int X;  
    if (IsEmpty (S))  
        printf ("Empty Stack");  
    else  
    {  
        X = S [Top];  
        Top = Top - 1;  
        printf("The element popped out is:%d",X);  
    }  
}
```

Routine to check empty stack

int IsEmpty(Stack S)

```
{  
    if (Top == -1)  
        return (1);  
  
    else  
        return(0);  
}
```

Routine to return top element of the stack

int TopElement (Stack S)

```
{  
    if (! IsEmpty (s))  
        return S[Top];  
    else  
        printf ("Empty Stack");  
return 0;  
}
```



S[Top] = 8

ADVANTAGE

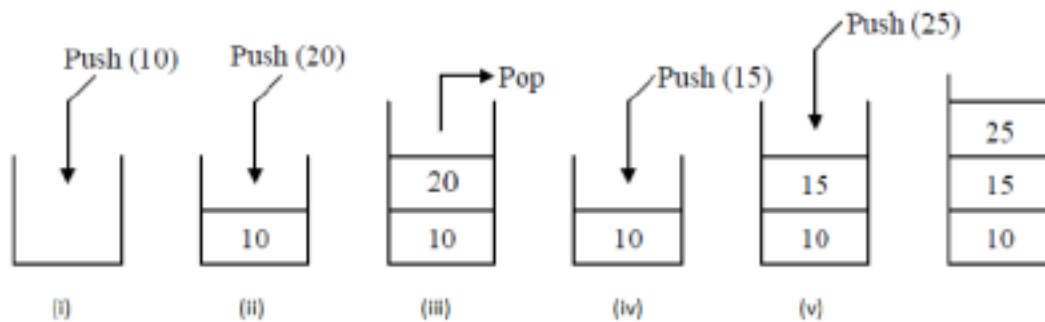
Can avoid pointers

DISADVANTAGE

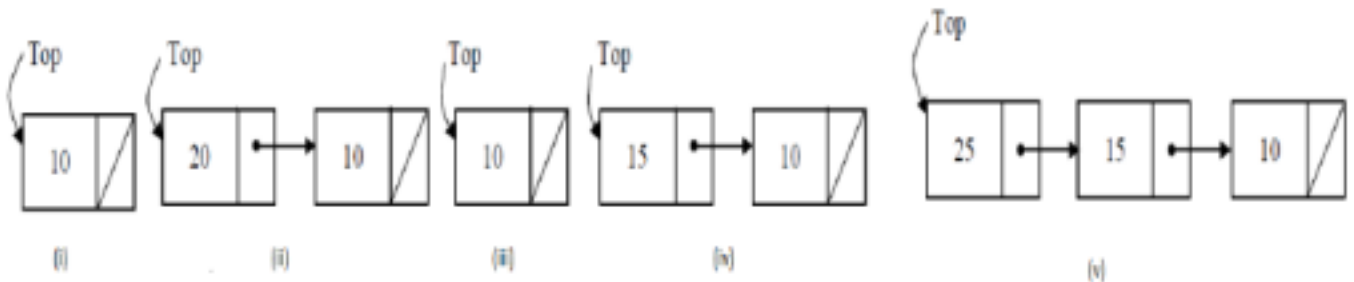
Need to declare an array size in advance.

Linked List Implementation of Stack

- Front of the list is pointed by Top.
- Push operation is performed by inserting an element at the front. •
- Pop operation is performed by deleting at the front.
- Top operation returns the element at the front.



Meenakshi Page 5



Declaration for Linked List Implementation

Struct Node

```
{
    int Element ;
    Struct Node *Next;
};
```

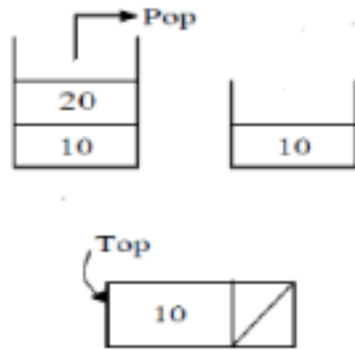
```
typedef Struct Node *Top;
```

Routine to push an element into stack

```
void push (int X, Stack Top)
```

```
{
    Struct Node * Temp;
    Temp = malloc (sizeof (Struct Node));
    if (Temp!= NULL)
    {
        Temp ->Element = X;
        Temp ->Next = Top;
        Top= Temp;
    }
}
```

Routine to pop an element from stack



Meenakshi Page 6

```
void pop (Stack Top)
```

```
{
```

```
Struct Node *Temp;
```

```
if (IsEmpty (Top))
```

```
    printf (“Empty Stack”);
```

```
else
```

```
{
```

```
    Temp = Top;
```

```
    Top= Top ->Next;
```

```
    free (Temp);
```

```
}
```

```
}
```

Routine to check empty stack

```
int IsEmpty (Stack Top)
```

```
{
```

```
    if (Top== NULL)
```

```
        return (1);
```

```
    else
```

```
        return(0);
```

```
}
```

Routine to return top element of a stack

```

int Top (Stack Top)
{
    if (! IsEmpty (Top))
        return Top ->Element;
    else
        printf ("Empty Stack");
}

```

Meenakshi Page 7

2.3 APPLICATIONS

Some of the applications of Stack are

- * Evaluating arithmetic expression
- * Conversion of Infix expression to Postfix expression
- * Balancing the symbols
- * Towers of Hanoi
- * Function Calls

2.4 BALANCING SYMBOLS

- Stacks can be used to check whether the given expression has balanced symbols or not. The algorithm is very much useful in compilers.
- Every right brace, bracket, and parenthesis must correspond to their left counterparts. The sequence `[( )]` is legal, but `[( )]` is wrong. The simple algorithm uses a stack and is as follows.

Steps:

Read one character at a time and repeat steps 1 and 2 until it encounters the delimiter '#'.

Step 1 : If the character is an opening symbol, push it onto the stack.

Step 2 : If the character is a closing symbol,

2.1: If it has corresponding opening symbol in the stack, POP it from the stack.
Otherwise, report an error "Unbalanced".

2.2: If the stack is empty report an error "Unbalanced".

Step 3 : At the end, if the stack is not empty, report an error "Unbalanced".
Otherwise, report "Balanced".

Meenakshi Page 8

Example 1 : (a + b) #		
<u>Read Character</u>	<u>Stack</u>	<u>Operation</u>
(	(	Push
a	(	-
+	(	-
b	(	-
)		Pop
#		Stack is Empty
Report "Balanced"		

Example 2:
((A+B) + [C-D])

Not Balanced

Different Types of Notations to Represent Arithmetic Expressions:

There are 3 different ways of representing the algebraic expression. *

INFIX EXPRESSION

* POSTFIX EXPRESSION

* PREFIX EXPRESSION

INFIX EXPRESSION

- Operator appears between the two operands.

Meenakshi Page 9

For example : - $A / B + C$

POSTFIX EXPRESSION

- Operator appears after the two operands
- It is also called reverse polish notation.

For example : - $AB / C +$ [Infix: $((A/B) + C)$]

PREFIX EXPRESSION

- Operator appears before the two operands.
- It is also called as polish notation.

For example : - $+ / ABC$ [Infix : $((A/B) + C)$]

Infix	Prefix	Postfix
$A+B$	$+AB$	$AB+$
$A+B-C$	$-+ABC$	$AB+C-$
$(A+B)*C-D$	$-*+ABCD$	$AB+C*D$

2.5 EVALUATING ARITHMETIC EXPRESSION

- To evaluate an arithmetic expressions, first convert the given infix expression to postfix expression and then evaluate the postfix expression using stack.

Algorithm:

Read the postfix expression one character at a time until it encounters the delimiter „ $\#$ ”.

Step 1 : - If the character is an operand, push its associated value onto the stack.

Step 2 : - If the character is an operator, POP two values from the stack, apply the operator to them and push the result onto the stack. The first pop is the second operand and the second pop is the first operand.

Example 1:

Post fix Expression : AB* CDE/-+ #

Meenakshi Page 10

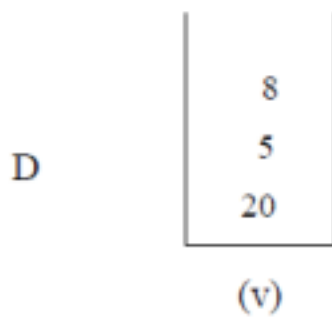
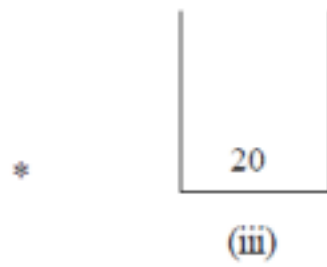
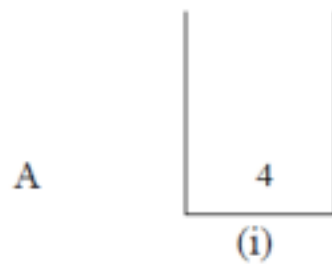
Let us consider the symbols A, B, C, D, E had the associated values :

Symbol	Value
A	4
B	5
C	5
D	8
E	2

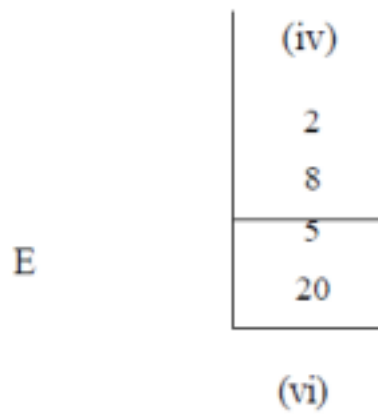
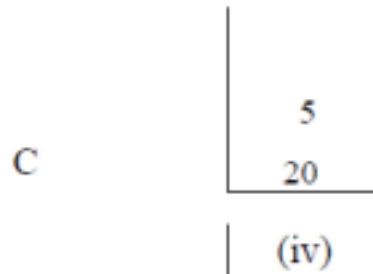
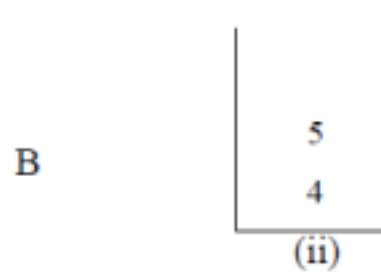
Solution:

From the given expression read one char

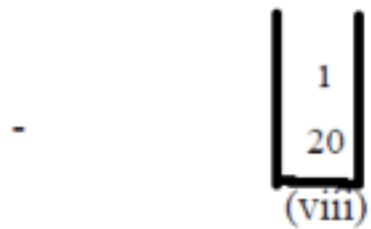
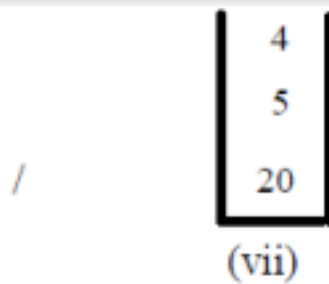
Read Character Stack



Input Character Stack



Meenakshi Page 11



end of the expression

Answer= 21

Example 2: **Input:** S = "123+*8-"

Solution: -3

2.6 Infix to Postfix Conversion

- Read the infix expression one character at a time until it encounters the delimiter. “#”.

1. If we have an opening parenthesis "(", we push it into the stack. 2. If we have an operand, we append it to our postfix expression. 3. If we have a closing parenthesis ")" we keep popping out elements from the top

of the stack and append them to our postfix expression until we encounter an opening parenthesis. We pop out the left parenthesis without appending it. 4. If we encounter an operator:-

- If the operator has higher precedence than the one on top of the stack (We can compare), we push it in the stack.
- If the operator has lower or equal precedence than the one on top of the stack, we keep popping out and appending it to the postfix expression. And the incoming operator is pushed in to the stack.

5. When the last token of infix expression has been scanned, we pop the remaining elements from stack and append them to our postfix expression.

Meenakshi Page 12

Example:

Infix Expression : $A * B + (C - D / E) \#$

Read Character

Stack

Output

A

A

*

*

A

B

*

AB

+

AB*





Solution: $AB * CDE / - +$

2.7 FUNCTION CALLS

- Another great use of stack is during the function call and return process.

- In Programming, whenever you make a call from one function to the function. The address of the calling function gets stored in the Stack. (PUSH)
- So, when the called function gets terminated. The program control move back to the calling function with the help of the address which was stored in the Stack. (POP)
- So, Stack plays the main role when it comes to calling a Function from other Function.

Example



Consider addrA, addrB, addrC be the addresses of the statements to which control is returned after completing the function A, B, and C, respectively.

A **Queue** is a linear data structure which follows **First In First Out (FIFO)** principle, in which **insertion** is performed at **rear** end and **deletion** is performed at **front** end.



Example: Waiting Line in Reservation Counter, queue in a cinema ticket counter, Scheduling of jobs in computer

2.9 OPERATIONS ON QUEUE

The fundamental operations performed on queue are

1. Enqueue
2. Dequeue

ENQUEUE:

- The process of inserting an element in the queue.

DEQUEUE:

- The process of deleting an element from the queue.

EXCEPTION CONDITIONS

Overflow: Attempt to insert an element, when the queue is full is said to be overflow condition.

Underflow: Attempt to delete an element from the queue, when the queue is empty is said to be underflow.

IMPLEMENTATION OF QUEUE

Queue can be implemented using array and linked list.

ARRAY IMPLEMENTATION

- In array implementation queue Q is associated with two pointers namely Rear and Front.

- For an empty queue Front = -1 and Rear = -1

- To insert an element X onto the queue Q, the Rear is incremented by 1 and then set $Q[\text{Rear}] = X$
- To delete an element, the $Q[\text{front}]$ is returned and the Front is incremented by 1.



Routine to check whether the queue is empty

int IsEmpty (Queue Q)

```
{
    if (Front<0)
        return (1);
    else
        return(0);
}
```

Routine to Enqueue

void Enqueue (int X,Queue Q)

```

{
    if (Rear+1 >= QueueSize)
        printf (" Queue overflow");
    else
    {
        Rear = Rear + 1;
        Q[Rear] = X;
        if(Front == -1)
            Front=0;
    }
}

```

Routine for Dequeue

```

void Dequeue ( Queue Q)
{
    int X;
    if (Front < 0)
        printf (" Queue Underflow");
    else
    {
        X = Q[Front];
        if (Front == Rear)
        {
            Front = -1;
            Rear = -1;
        }
        else
            Front = Front + 1 ;
    }
}

```

```

    }
}

```

Linked List Implementation of Queue

- Enqueue operation is performed at the end of the list (Rear end) •

Dequeue operation is performed at the front of the list (Front end)

Declaration for linked list implementation of queue ADT struct Node

```

{
    int Element;
    struct Node *Next;
}* Front = NULL, *Rear = NULL;
typedef struct Node * Queue;

```

Routine to check whether the queue is empty

```

int IsEmpty (Queue Q)
{
    if (Front == NULL)
        return (1);
    else
        return(0);
}

```

Routine to enqueue an element in queue

```

void Enqueue (int X)
{
    struct node *newnode;
    newnode = malloc (sizeof (struct node));
    if(Rear == NULL)
    {
        newnode ->Element = X;

```

```

        Front = newnode;
        Rear = newnode;
    }
else
{
    newnode ->Element = X;
    newnode ->Next = NULL;
    Rear ->next = newnode;
    Rear = newnode;
}
}

```

2.10 CIRCULAR QUEUE

Circular Queue Model

- In Circular Queue, elements are arranged in circular fashion. •
- So the first element comes just after the last element.
- If the last location of the queue is full, Insertion of a new element is performed at the very first location of the queue

It overcomes the problem of unutilized space in linear queues, when it is implemented as arrays.

Circular Queue Operations

- To perform the insertion of an element to the queue, the position of the element is calculated by

$\text{Rear} = (\text{Rear} + 1) \% \text{Maxsize}$. and then set $\text{CQ}[\text{Rear}] = \text{value}$.

- To perform the deletion, element is taken from Front

$\text{Value} = \text{CQ}[\text{Front}]$ and the position of the Front pointer is calculated by $\text{Front} = (\text{Front} + 1) \% \text{maxsize}$.

Routine to insert an element into Circular Queue

```
void CEnqueue (int X,CQueue CQ)
```

```
{
```

```
    if ((Rear + 1) % Maxsize == Front)
```

```
        printf ("Queue overflow");
```

```
Meenakshi Page 21
```

```
    else
```

```
{
```

```

        Rear = (Rear + 1)% Maxsize;
        CQ[Rear] = X;

        if (Front == -1)
            Front = 0;
    }
}

void CDequeue (CQueue CQ )
{
    int X;
    if (front < 0)
        printf("Queue underflow");
    else
    {
        X = CQ[Front];
        if(Front==Rear)
        {
            Front=-1;
            Rear=-1;
        }
        else
        {
            Front = (Front + 1) % maxsize;
            printf("The element deleted is %d",X);
        }
    }
}
}

```

In Double Ended Queue, insertion and deletion operations are performed at both the ends.

DEQUEUE

INPUT RESTRICTED DEQUEUE

In this type insertions are allowed at one end and deletions are allowed at both ends.

OUTPUT RESTRICTED DEQUEUE

In this type deletions are allowed at one end and insertions are allowed at both ends.

Operations performed on dequeue

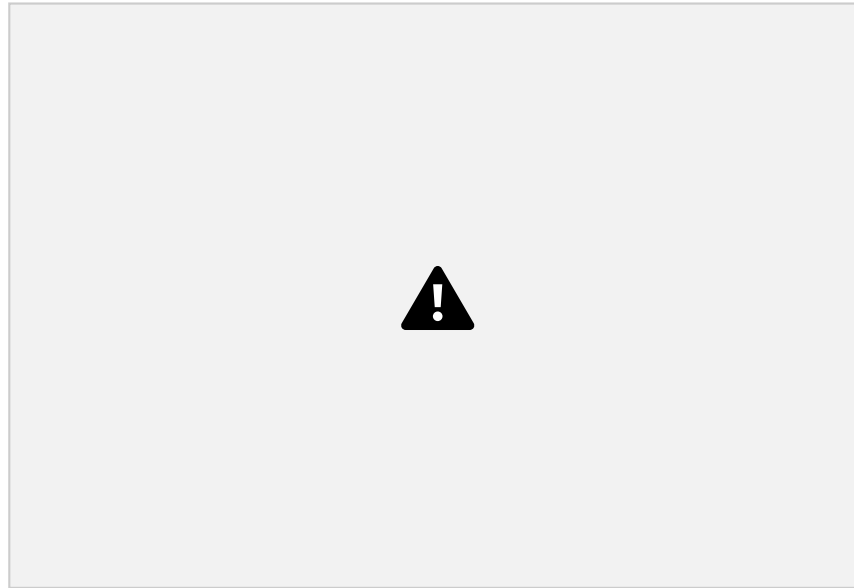
There are the following operations that can be applied on a dequeue

- Insertion at front
- Insertion at rear
- Deletion at front

- Deletion at rear

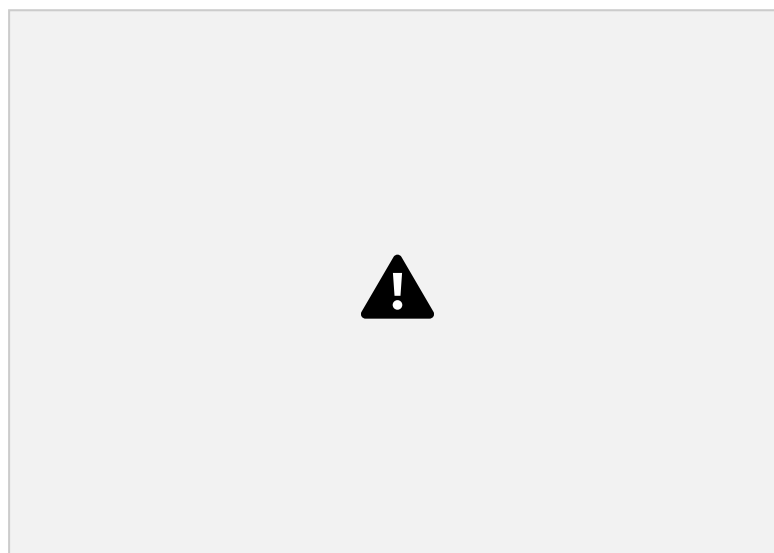
Insertion at the front end

- If the queue is empty, both rear and front are initialized with 0. Now, both will point to the first element.
- Otherwise, check the position of the front if the front is less than 1 ($\text{front} < 1$), then reinitialize it by $\text{front} = n - 1$, i.e., the last index of the array.



Insertion at the rear end

- If the queue is empty, both rear and front are initialized with 0. Now, both will point to the first element.
- Otherwise, increment the rear by 1. If the rear is at last index (or $\text{size} - 1$), then instead of increasing it by 1, we have to make it equal to 0.



- If the queue is empty, i.e., $\text{front} = -1$, it is the underflow condition, and we cannot perform the deletion. If the queue is not full, then the element can be inserted from the front end by using the below conditions -
- If the deque has only one element, set $\text{rear} = -1$ and $\text{front} = -1$.
- Else if front is at end (that means $\text{front} = \text{size} - 1$), set $\text{front} = 0$.
- Else increment the front by 1, (i.e., $\text{front} = \text{front} + 1$).



Deletion at the rear end

In this operation, the element is deleted from the rear end of the queue. Before implementing the operation, we first have to check whether the queue is empty or not.

If the queue is empty, i.e., $\text{front} = -1$, it is the underflow condition, and we cannot perform the deletion.

If the deque has only one element, set $\text{rear} = -1$ and $\text{front} =$

-1. If $\text{rear} = 0$ (rear is at front), then set $\text{rear} = n - 1$.

Else, decrement the rear by 1 (or, $\text{rear} = \text{rear} - 1$).



2.12 APPLICATIONS OF QUEUE

- Batch processing in an operating system
- To implement Priority Queues and Priority Queues can be used to sort the elements using Heap Sort.
- Simulation.
- Mathematics Queuing theory.
- Computer networks
- Job scheduling in computers
- When jobs are submitted to a printer, they are arranged in order of arrival.

Thus, essentially, jobs sent to a line printer are placed on a queue. • Calls to large companies are generally placed on a queue when all operators are busy